

1. Package Installation & Management

Modern Linux package managers automate software acquisition, cryptographic signature verification, dependency resolution, and system upgrades. In DevOps workflows, automated provisioning scripts (Ansible, cloud-init, and Docker builds) rely directly on resilient package manager commands.

Operation	Debian / Ubuntu (apt)	RHEL / Rocky / Fedora (dnf)	Alpine Linux (apk)
Update Catalog Index	<code>sudo apt-get update -y</code>	<code>sudo dnf check update</code>	<code>apk update</code>
Install Package	<code>sudo apt-get install -y <pkg></code>	<code>sudo dnf install -y <pkg></code>	<code>apk add --no-cache <pkg></code>
Purge / Remove	<code>sudo apt-get purge -y <pkg></code>	<code>sudo dnf remove -y <pkg></code>	<code>apk del <pkg></code>
Full Upgrade	<code>sudo apt-get upgrade -y</code>	<code>sudo dnf upgrade -y</code>	<code>apk upgrade</code>
Clean Cache	<code>sudo apt-get clean && rm -rf /var/lib/apt/lists/*</code>	<code>sudo dnf clean all</code>	<code>rm -rf /var/cache/apk/*</code>
List Installed Files	<code>dpkg -L <pkg></code>	<code>rpm -ql <pkg></code>	<code>apk info -L <pkg></code>

Standard Package Installation Sequence

STEP 1 Synchronize Package Repository Metadata

Update local index caches before attempting installs to ensure package hashes and mirror trees match:

```
sudo apt-get update -y
```

STEP 2 Install Target Packages with Non-Interactive Flags

Supply the automatic affirmative response flag (`-y`) and export frontend suppression variables to eliminate CI/CD pipeline hangs:

```
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends nginx curl jq
```

STEP 3 Verify Installation & Clean Workspace

Confirm binary presence and prune cached `.deb` archives to conserve disk capacity:

```
which nginx && nginx -v
sudo apt-get clean && sudo rm -rf /var/lib/apt/lists/*
```

DevOps Containerization Rule: Dockerfile Layer Hygiene

Always consolidate `update` , `install` , and `clean` into a single `RUN` statement within Dockerfiles. Separate steps cause deleted archive files to remain permanently embedded inside lower filesystem layers.

2. Linux Cron Jobs — Complete Automation Guide

Cron is the time-based job scheduler in Unix-like operating systems. It executes commands or shell scripts in the background at fixed intervals, making it a critical tool for infrastructure maintenance.

Primary DevOps Applications:

- Automated incremental and full backup routines
- Log rotation, compaction, and disk retention cleanup
- Periodic TLS certificate verification and renewal checks
- Database dump scripts and snapshot uploads to object stores
- System health checks, polling daemons, and alert monitors

2.1 Daemon Verification & Initialization

Verify that the scheduling daemon is active and enabled across system restarts:

```
# Ubuntu / Debian
sudo systemctl enable --now cron
systemctl status cron

# RHEL / CentOS / Rocky Linux
sudo systemctl enable --now crond
systemctl status crond
```

2.2 Crontab Management Commands

Command	Operation	Operational Description
<code>crontab -e</code>	Edit crontab	Opens the current user's crontab in default text editor (nano/vim).
<code>crontab -l</code>	List crontab	Dumps all currently active cron jobs for the executing account.
<code>crontab -r</code>	Remove crontab	Caution: Permanently deletes the user's active crontab file without prompting.
<code>sudo crontab -u deploy -e</code>	Manage target user	Enables root/DevOps engineers to modify a service account's crontab.

2.3 Cron Time Syntax & Field Breakdown

Linux Systems Engineering & DevOps Administration Handbook Page 2 of 6

Common Production Cron Patterns:

Schedule Goal	Cron Expression	Example Task Execution
Every minute	* * * * *	* * * * * /opt/scripts/heartbeat.sh
Every 5 minutes	* /5 * * * *	* /5 * * * * /usr/bin/python3 /opt/metrics.py
Every hour on the hour	0 * * * *	0 * * * * /usr/bin/certbot renew --quiet
Every night at 2:00 AM	0 2 * * *	0 2 * * * /home/vagrant/backup.sh
Every Monday at 8:00 AM	0 8 * * 1	0 8 * * 1 /opt/reports/weekly_summary.sh
At system reboot	@reboot	@reboot /opt/scripts/node_init.sh

2.4 Step-by-Step Script Automation Workflow

STEP 1 Author Production Shell Script

Create a dedicated script file (`~/backup.sh`) with explicit error handling and absolute paths:

STEP 2 Assign Execution Permissions & Test Interactively

Ensure the executable bit is toggled and validate normal operation prior to scheduling:

STEP 3 Configure Crontab with Output Redirection and Concurrency Locking

Launch `crontab -e` and append the entry with `flock` and standard stream redirection:

```
0 2 * * * /usr/bin/flock -n /tmp/backup.lock /bin/bash /home/vagrant/backup.sh >> /home/vagrant/backups/backup.log 2>&1
```

DevOps Cron Rules

- **Minimal Environment (\$PATH):** Cron executes with a barren environment and does not load `~/.bashrc`. Explicitly declare `PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin` at top of crontab.
- **Overlap Prevention:** Use `flock -n` to protect servers against stacked, resource-exhausting processes if a previous run is still active.
- **Stream Redirection:** Always redirect `stdout` and `stderr` (`>> /path/to/log 2>&1`) to retain operational trace logs.

3. Service Management with systemctl & Systemd

Systemd is the standard init system and process supervisor for enterprise Linux distributions. It manages process lifecycles, service dependency graphs, and resource constraints via declarative unit configuration files.

3.1 Essential Service Administration Commands

```
# Lifecycle Control
sudo systemctl start <service> # Launch service immediately
sudo systemctl stop <service> # Terminate service (SIGTERM followed by SIGKILL)
sudo systemctl restart <service> # Stop and relaunch process
sudo systemctl reload <service> # Reload configuration files without dropping open connections

# Boot & Persistence Management
sudo systemctl enable <service> # Create symlinks in systemd target directory for boot startup
sudo systemctl enable --now <service> # Enable and launch immediately in a single transaction
sudo systemctl disable <service> # Remove startup symlinks

# Inspection & Diagnostics
systemctl status <service> # Display PID, status, memory consumption, and recent log tail
systemctl is-active <service> # Returns 'active' or 'inactive'
systemctl is-enabled <service> # Returns 'enabled' or 'disabled'
systemctl --failed # Audit all failed/degraded units across the operating system
sudo systemctl daemon-reload # Re-read and compile modified unit files from disk
```

3.2 Creating a Production-Ready Systemd Unit Step-by-Step

STEP 1 Provision Dedicated System Service Account

Follow the principle of least privilege by running the application under an unprivileged system user:

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin appuser
```

STEP 2 Draft Unit File (/etc/systemd/system/api-service.service)

Create the unit specification containing dependency definitions, execution parameters, and security sandboxing:

```
[Unit]
Description=Production API Service
```

```
Documentation=https://internal-wiki.company.local/docs/api
After=network.target remote-fs.target
Wants=network-online.target
```

```
[Service]
Type=simple
User=appuser
Group=appuser
```

```
WorkingDirectory=/var/www/api
ExecStart=/usr/bin/python3 -m uvicorn main:app --host 0.0.0.0 --port 8000
ExecReload=/bin/kill -s HUP $MAINPID
Restart=always
RestartSec=5s
```

```
# Security Hardening Directives
ProtectSystem=full
ProtectHome=true
NoNewPrivileges=true
PrivateTmp=true
```

```
# Resource Boundaries & Descriptors
LimitNOFILE=65535
MemoryMax=1G
CPUQuota=150%
```

```
[Install]
WantedBy=multi-user.target
```

STEP 3 Register, Enable, and Start Service

Inform systemd of the new unit file and trigger startup:

```
sudo systemctl daemon-reload
sudo systemctl enable --now api-service.service
systemctl status api-service.service
```

STEP 4 Log Auditing with journalctl

Query aggregated system journal logs for the application unit:

```
# Follow live logs (similar to tail -f)
journalctl -u api-service.service -f

# Filter logs since current boot
journalctl -u api-service.service -b

# Filter priority errors only
journalctl -u api-service.service -p err..emerg --no-pager
```

4. Essential DevOps Infrastructure & Diagnostic Concepts

Beyond basic service administration, production engineering requires operational familiarity with port auditing, process troubleshooting, and capacity monitoring.

Engineering Domain	Diagnostic Command	Operational Purpose
--------------------	--------------------	---------------------

Network Sockets	<code>sudo ss -tulpn</code>	Audit listening TCP/UDP ports and associated process PIDs. Replaces <code>netstat</code> .
Network Probing	<code>nc -zv 10.0.0.15 443</code>	Verify network firewall reachability and TCP socket handshakes without client overhead.

Engineering Domain	Diagnostic Command	Operational Purpose
Filesystem Inodes	<code>df -i</code>	Audit inode consumption. Disks can report 0% storage capacity if millions of tiny files exhaust inodes.
Memory & OOM	<code>sudo dmesg -T grep -i oom</code>	Inspect kernel rings to verify if the Linux Out-Of-Memory subsystem terminated your daemon.
Storage I/O Bottlenecks	<code>iostat -xz 1 5</code>	Evaluate device saturation (<code>%util</code>) and I/O wait times to diagnose slow disk queues.